

BDF - Customized Product Documentation

This document provides customized product documentation for BDF.

BDF Bakery Automation System Documentation Index

This folder contains detailed reference material for your BDF automated inspection system.

Documents

- [System Overview: Purpose and Capabilities](#system-overview-purpose-and-capabilities)
 - Explains the system's role, operating modes, and what it can do for your bakery.
- [System Structure: How Your BDF System Works](#system-structure-how-your-bdf-system-works)
 - Describes the internal organization, camera options, and image processing for optimal production monitoring.
- [System Settings: Customizing Your BDF System](#system-settings-customizing-your-bdf-system)
 - Guides you through `.env` and `settings.json` for configuring system behavior, including cameras and AI.
- [API Reference: Connecting to Your BDF System](#api-reference-connecting-to-your-bdf-system)
 - Provides details for integrating with the main system and dedicated camera service.
- [Deployment Options: Setting Up Your BDF System](#deployment-options-setting-up-your-bdf-system)
 - Offers guidance for local, debug, remote, and containerized installations.
- [Core Technologies: Building Blocks of Your BDF System](#core-technologies-building-blocks-of-your-bdf-system)
 - Briefly outlines the internal camera, AI, and color analysis libraries.
- [Troubleshooting Guide: Resolving Common Issues](#troubleshooting-guide-resolving-common-issues)
 - Helps diagnose and resolve typical operational problems with your BDF system.

System Overview: Purpose and Capabilities

Purpose

The BDF Bakery Automation System is designed for line-scan acquisition, image stitching, live streaming, and optional AI-assisted inspection of your bakery products. It uses two Basler line cameras to ensure comprehensive quality control and supports flexible workflows for both production and development environments.

Core Capabilities

Your BDF system offers key features to enhance your bakery automation:

- **Dual-Camera Imaging and Stitching:** High-speed camera acquisition combines multiple camera views into a single, complete image of your product for thorough inspection.
- ***Space for Diagram:** Illustrate two cameras capturing a product on a line, then the stitched output image.*
- **Live Product Feed:** Browser-friendly JPEG capture and live streaming via WebSocket provide real-time visual monitoring of your production line.
- **Flexible Camera Operations:**
- **Direct Integration (Local):** The main BDF system directly controls the Basler cameras.
- **Separate Camera Unit (Remote):** A dedicated camera service runs closer to the production line, sending images to the main BDF system.
- **Simulation Mode (Emulated):** Generates synthetic images for testing, training, or UI validation without needing physical cameras.
- **Optional AI-Powered Inspection:** Integrate advanced AI models (RF-DETR or YOLO) for automated defect detection and anomaly tracking.
- **Automated Color Analysis:** Tracks and analyzes product colors, providing historical data and real-time feedback for consistent quality control.
- **External Analysis Integration:** Ability to forward product images to external systems for additional analysis or quality checks.
- **On-Demand Image Analysis:** Analyze specific product images from your local storage using the configured AI pipeline.

Operating Modes

The BDF system can operate in three primary camera modes:

- **`local` Mode:** The main system directly manages the Basler cameras for a compact, integrated setup.
- **`remote` Mode:** The main system communicates with a separate, dedicated camera service (typically running on the production line) over your network.
- **`emulated` Mode:** The system generates simulated product images internally, ideal for testing the user interface or developing AI models without physical camera hardware.

For AI processing, you have two options:

- **Disabled:** The system provides raw product images and streaming only, without AI inspection.
- **Enabled:** AI models (RF-DETR or YOLO with SORT tracking) perform automated inspection and analysis.

High-Level Flow

1. The BDF system application starts.
2. Based on your settings, a camera runtime is initialized (local, remote, or emulated).
3. The chosen camera runtime begins capturing, polling, or simulating product images.
4. If AI inspection is enabled, the AI models start loading in the background.
5. The latest product image is made available for:
 - Viewing via a web browser.
 - Retrieving through an API call.
 - Sending to external analysis services.
 - Live streaming via WebSocket.
6. If color analysis is active, relevant product samples are buffered and accessible through specific color endpoints.

Main Components

- **System Services (`src/server`):** Manages the application lifecycle, exposes data endpoints, selects camera operation modes, and reports AI status.
- **Operator Interface (`src/front`):** Provides the web-based viewer and health dashboard for monitoring.
- **Camera Imaging (`src/libs/racer2\_camera`):** Handles camera acquisition, image stitching, resizing, and preparation of product images.
- **AI Inspection (`src/libs/ai`):** Manages AI models, performs detection, tracking, and annotation for automated inspection.
- **Color Analysis (`src/libs/color`):** Provides color conversion, per-object analysis, and buffering of color data.
- **Documentation (`Docs`):** This collection of operational and technical information.

System Split Options

Your BDF system can run as a single process or as two cooperating services:

- **Main Application:** On network port `8000` (e.g., for AI, web UI, and data integration).
- **Optional Dedicated Camera Service:** On port `8001` (e.g., for direct camera control on the production line).

This split is beneficial when the cameras need to be physically close to the Basler hardware on the production line, while the main web or AI processing runs on a separate machine.

- *Space for Diagram: Illustrate the split deployment, with cameras connected to one unit, sending data to another unit running the main app.*

System Structure: How Your BDF System Works

This section explains the core components and processes within your BDF system, focusing on how product images are acquired, processed, and analyzed for bakery automation.

System Services

The BDF system is organized into functional layers:

1. Starting the System:

- ``main.py`` launches the main application, typically on port ``8000``.
- ``camera_main.py`` launches the dedicated camera service, typically on port ``8001``.

2. Core Services Layer:

- ``src/server/app.py`` provides the main operator interface, live streaming, AI inspection, and color analysis endpoints.
- ``src/server/camera_service.py`` offers the camera-only API for the dedicated camera unit.

3. Core Function Libraries Layer:

- ``src/libs/racer2_camera`` handles camera imaging and stitching.
- ``src/libs/ai`` manages automated inspection and tracking.
- ``src/libs/color`` performs color analysis.

Camera Operation Modes

The main BDF application can configure its camera input in ``src/server/app.py`` using these modes:

- **``local` Mode:`**
 - Uses ``src/server/local_camera_runtime.py``.
 - The BDF system directly accesses Basler cameras within its own process.
- **``remote` Mode:`**
 - Uses ``src/server/remote_camera_runtime.py``.
 - Product images and alignment data are fetched from a separate camera service (the dedicated camera unit).
- **``emulated` Mode:`**
 - Uses ``src/server/emulated_camera_runtime.py``.
 - Synthetic product images are generated internally for development, testing, and training purposes.

The dedicated camera service (running via `camera_main.py`) only supports `local` and `emulated` modes, as it does not host AI inspection features itself.

Product Imaging Pipeline

The live image acquisition process is managed in `src/libs/racer2_camera/src/line_camera.py`.

- **Space for Diagram: Illustrate the camera pipeline steps as a flow chart.**

Here's a high-level flow for how your BDF system captures and prepares product images:

1. **Camera Discovery:** The system identifies connected Basler cameras.
2. **Camera Setup:** It verifies the configured number of cameras and sets up essential features like triggers, pixel format, and optional white balance (unless externally managed).
3. **Strip Acquisition:** The cameras capture the configured number of "strips" (thin slices) of each product.
4. **Vertical Stacking:** These strips are combined vertically for each camera.
5. **Alignment Adjustment:** Horizontal and vertical offsets are applied to ensure proper positioning.
6. **Image Stitching:** Outputs from multiple cameras are seamlessly stitched together to form a single, complete image of the product.
7. **Color Conversion:** The final image is converted to BGR format if needed, preparing it for display and AI processing.
8. **Resizing and Encoding:** The image is resized and converted into a JPEG format for efficient storage and transmission.
9. **Image Cache:** The latest JPEG product image is stored for quick access by various consumers (e.g., web UI, other systems).

Runtime Alignment Changes: Operators or integrated systems can adjust image stitching alignment through specific API endpoints:

- `GET /camera/alignment`: To retrieve current alignment settings.
- `POST /camera/alignment`: To update alignment settings.

Remote Camera Data Flow

When using the `remote` camera mode, `src/server/remote_camera_runtime.py` continuously retrieves data from the dedicated camera service (defined by `REMOTE_CAMERA_BASE_URL`):

- `/status`: Checks the health and readiness of the camera service.
- `/capture`: Fetches the latest product image.
- `/camera/alignment`: Retrieves alignment settings.

This setup allows for flexible deployments where:

- The camera unit maintains direct connection to Basler hardware on the production line.
- The main BDF system consumes product images efficiently over your network.
- The web interface and AI inspection can be deployed separately from the cameras.

Automated Inspection (AI) Pipeline

The AI process for automated product inspection is managed in ``src/libs/ai/pipeline.py``.

The selected AI model is controlled by the ``AI_MODEL_NAME`` setting:

- ``rfdetr``: Provides RF-DETR-backed detection and tracking of product anomalies.
- **Any other value:** Uses YOLO detection combined with SORT tracking.

Important Operational Behavior:

- **Background Initialization:** AI model setup runs in the background, ensuring the BDF system starts up quickly without delays.
- **AI Readiness Monitoring:** The ``/status`` endpoint reports the progress of AI model initialization.
- **Live Feed Transition:** The WebSocket stream will initially show raw product images until the AI is fully ready to perform inspections.
- **On-Demand Analysis:** The ``run_single_image_inference()`` function powers the ``POST /inference`` endpoint for analyzing specific product image files.

Color Analysis

The AI pipeline also includes a ``ColorBuffer`` that stores a recent history of tracked objects and their color data.

Color features include:

- **Latest Sample State:** Provides the most recent color data for each tracked object.
- **Rolling History:** Maintains a short history of color changes for each object.
- **CSV Export:** Allows you to export color data for further analysis.
- **Product-Shaped Output:** Delivers color analysis results in a format suitable for integration with downstream production monitoring systems.

Accessing Product Data

The latest processed product image can be accessed by:

- ``GET /capture``: For retrieving the image directly.
- ``POST /analyze``: For forwarding the image to external analysis services.
- ``POST /inference``: For performing on-demand AI analysis on local files.
- ``WebSocket /ws``: For live streaming in the browser.
- The operator interface in your web browser.

System Resilience

The BDF system is designed for high availability and continuous operation:

- **Soft Failure:** Startup continues even if cameras are temporarily unavailable, allowing other parts of the system to function.
- **Continuous Status:** The `/status` endpoint remains available even if image capture fails, providing diagnostic information.
- **Structured Errors:** Endpoints like `/capture`, `/analyze`, and `/inference` return clear error messages when issues occur, rather than crashing the system.
- **Client Communication:** WebSocket clients receive informative text messages (e.g., `warming_up`, camera error descriptions) instead of unexpected disconnections.

System Settings: Customizing Your BDF System

Your BDF system uses configuration files to manage its runtime behavior and sensitive information. Understanding these sources helps you tailor the system to your specific bakery automation needs.

Configuration Sources

The BDF system uses two primary types of configuration:

- **Runtime Settings (`settings.json`):** Non-secret settings that control how the system operates, such as camera modes, AI preferences, and delays.
- **Sensitive Information (`.env` files):** Stores secrets like API keys and environment selection (e.g., debug mode).

The configuration loader is located in `src/settings.py`.

Loading Order

The system resolves configuration values in a specific order:

1. **`.env`:** This file is loaded first, typically containing base environment variables and secrets.
2. **`APP_ENV_FILE` (Optional):** If the `APP_ENV_FILE` variable is set (e.g., to `.env.debug`), that specific file is loaded on top of `.env`.
3. **`.env.debug` (Conditional):** If `APP_ENV` is set to `debug` and `APP_ENV_FILE` is not specified, `.env.debug` is loaded over `.env`.
4. **`settings.json`:** This file is loaded after all `.env` files.
5. **Overrides:** If a setting appears in both an environment file and `settings.json`, the value from the environment file (or a later-loaded environment file) always takes precedence.

In Practice for BDF:

- **Secrets** (e.g., API keys for external services) should always be kept in `.env` files.
- **Profile-specific non-secrets** (e.g., `CAMERA_MODE` for a debug environment) typically live in `settings.json`.
- **Environment variables** can temporarily override any setting from `settings.json` or `.env` when needed.

settings.json

`settings.json` supports different operational profiles for your BDF system:

- `production`: Settings for live production environments.
- `debug`: Settings for development and testing.
- `docker`: Settings specifically for containerized deployments.

Here are some key settings used by the BDF application:

Live Feed Settings (`Stream`)

- `STREAM_DELAY_S`: The pause between sending product images over the WebSocket live feed.

External Analysis Integration (`External Inference`)

- `INFERENCE_URL`: The network address of an external service for advanced product analysis.
- `INFERENCE_RETRIES`: Number of attempts to connect to the external analysis service.
- `INFERENCE_RETRY_DELAY_S`: Pause between retry attempts for external analysis.

Camera Operation Modes (`Camera Runtime`)

- `CAMERA_MODE`: Defines how the BDF system interacts with cameras:
- `local`: The main BDF system directly controls Basler cameras.
- `remote`: The main BDF system retrieves images from a separate camera service.
- `emulated`: The BDF system generates synthetic product images for testing.
- `CAMERA_WHITE_BALANCE`: When `true`, enables an automatic white balance calibration after cameras start.
- `CAMERA_TRIGGER_SOURCE`: Default: `Line2`. Specifies the trigger input for cameras.
- `CAMERA_TRIGGER_ACTIVATION`: Default: `RisingEdge`. Defines when the trigger activates (e.g., on a rising signal).
- `CAMERA_TRIGGER_SELECTOR`: Default: `LineStart`. Selects which trigger feature to configure.
- `CAMERA_TRIGGER_MODE`: Boolean setting; `true` activates camera triggering, `false` deactivates it.
- `CAMERA_PIXEL_FORMAT`: The format in which camera images are processed. Supported: `BGR8`, `BGR8Packed`, `YCbCr422_8`, `YUV422Packed`, `YUV422_YUYV_Packed`.
- `BGR8` and `BGR8Packed` are treated as ready-to-use BGR images for OpenCV.
- YUV422 formats are automatically converted to BGR for processing and display.
- `CAMERA_CONFIG_BY_PYLON`: When `true`, the BDF system will *not* write camera configuration (like pixel format, exposure) and will rely on settings configured externally (e.g., in Basler Pylon Viewer).

- ``REMOTE_CAMERA_BASE_URL``: The network address of the dedicated camera service when ``CAMERA_MODE`` is ``remote``.
- ``REMOTE_CAMERA_TIMEOUT_S``: Maximum time to wait for a response from the remote camera service.
- ``REMOTE_CAMERA_POLL_INTERVAL_S``: How often the main BDF system checks for new images from the remote camera service.
- ``EMULATED_CAMERA_WIDTH``: Width of synthetic images in ``emulated`` mode.
- ``EMULATED_CAMERA_HEIGHT``: Height of synthetic images in ``emulated`` mode.
- ``EMULATED_CAMERA_FPS``: Frames per second for synthetic images in ``emulated`` mode.
- ``EMULATED_CAMERA_JPEG_QUALITY``: JPEG quality for synthetic images.
- ``EMULATED_CAMERA_IMAGE_PATH``: Optional path to a source image file used by the emulated runtime instead of generating frames.

Automated Inspection Settings (``AI Runtime``)

- ``DETECTION_AND_TRACK``: When ``true``, enables both object detection and tracking for AI inspection.
- ``AI_MODEL_NAME``: Specifies which AI model to use (e.g., ``rfdetr`` for RF-DETR-based inspection, or other values for YOLO with SORT).
- ``YOLO_MODEL_PATH``: File path to the YOLO model (if selected).
- ``RFDETR_CONF_THRESH``: Confidence threshold for RF-DETR detections (e.g., 0.5 means 50% confidence).
- ``CORE_MODEL_SAM_ENABLED``: Enables specific segmentation model features.
- ``CORE_MODEL_SAM3_ENABLED``: Enables specific segmentation model features.
- ``CORE_MODEL_GAZE_ENABLED``: Enables specific attention model features.

.env Files

These files contain sensitive and environment-specific keys for your BDF system.

Current keys that typically hold secrets:

- ``INFERENCE_API_HEADER``: HTTP header name for external inference service authentication.
- ``INFERENCE_API_KEY``: API key for external inference service authentication.
- ``RFDETR_MODEL_ID``: Identifier for the RF-DETR model.
- ``ROBOFLOW_API_KEY``: API key for Roboflow services (if used).

Example ``env``:

```
``env
```

```
INFERENCE_API_HEADER=X-API-Key
```

```
INFERENCE_API_KEY=your-secret-api-key
```

```
RFDETR_MODEL_ID=bdf-bakery-project/v1
```

```
ROBOFLOW_API_KEY=your-roboflow-api-key
```

...

Useful settings for environment selection:

```
env
```

```
APP_ENV=debug
```

```
APP_ENV_FILE=.env.debug
```

...

These allow you to quickly switch between different operational environments for your BDF system.

Runtime Settings Access

The BDF application makes all resolved configuration settings available through a `settings` object in `src/settings.py`.

This `settings` object is used by:

- `src/server/app.py`: Main application services.
- `src/server/camera_service.py`: Dedicated camera service.
- Specific camera modules under `src/server` for camera runtime configuration.

Camera Library Configuration

The camera imaging library has its own specific configuration in `src/libs/racer2_camera/src/config.py`.

Key fields that impact camera operation for BDF:

- `num_cameras`: Number of Basler cameras connected.
- `num_images`: Number of strips acquired per camera per frame.
- `max_stack`: Maximum number of stacked strips.
- `pixels_per_shot`: Vertical resolution of each strip.
- `h_offset`: Horizontal alignment offset for stitching.
- `v_offset`: Vertical alignment offset for stitching.
- `exposure_time_us`: Camera exposure time in microseconds.
- `pixel_format`: Image format from the camera.
- `trigger_source`: Camera trigger input line.
- `trigger_activation`: Trigger activation edge.
- `trigger_selector`: Which trigger feature to control.
- `trigger_mode`: Enable or disable camera trigger.
- `white_balance`: Enable or disable automatic white balance.
- `is_config_by_pylon`: If `true`, camera settings are externally managed.
- `jpeg_quality`: Quality of output JPEG images.

Most of these are set internally by the BDF application. However, ``white_balance``, ``pixel_format``, trigger settings, and ``is_config_by_pylon`` are controlled by the ``CAMERA_*`` keys in ``settings.json`` or your environment variables.

Camera Pixel Format

Use ``CAMERA_PIXEL_FORMAT`` to specify how the BDF system interprets raw camera data after stitching:

```
"""json

"CAMERA_PIXEL_FORMAT": "BGR8"

"""
```

- ``BGR8`` and ``BGR8Packed``: Skip color conversion as pypylon provides images in OpenCV-compatible BGR channel order. Use this when your camera is already configured for BGR output.

```
"""json

"CAMERA_PIXEL_FORMAT": "YCbCr422_8"

"""
```

- ``YCbCr422_8`` and other YUV422 formats: These are automatically converted to BGR before resizing, JPEG encoding, streaming, and AI processing.

Pylon-Managed Camera Configuration

Set ``CAMERA_CONFIG_BY_PYLON`` to ``true`` if camera features (like pixel format, exposure, trigger, packet size, or height) are managed externally, for example, using Basler Pylon Viewer. The BDF system will then skip writing these settings during startup.

```
"""json

"CAMERA_CONFIG_BY_PYLON": true

"""
```

When this value is ``false``, the BDF system will configure camera features based on its internal ``Racer2CameraConfig`` settings during startup.

AI Library Configuration

The AI library config is located in ``src/libs/ai/config.py``.

Important fields for BDF's automated inspection:

- ``model_name``: The specific AI model to use (e.g., ``rfdetr``, ``yolo``).
- ``yolo_model_path``: Path to the YOLO model file.
- ``rfdetr_model_id``: ID for the RF-DETR model.

- ``roboflow_api_key``: API key for Roboflow access.
- ``rfdetr_conf_thresh``: Confidence threshold for RF-DETR detections.
- ``core_model_sam_enabled``, ``core_model_sam3_enabled``, ``core_model_gaze_enabled``: Enable/disable advanced model features.
- ``color_analyze_enabled``: When ``true``, enables color analysis within the AI pipeline.

The BDF main application currently always enables ``color_analyze_enabled`` within the AI configuration it uses.

Camera Mode

The ``CAMERA_MODE`` setting fundamentally changes how your BDF system acquires images:

- **``local``**: The main BDF system initializes Basler cameras directly using the ``pypylon`` library.
- **``remote``**: The main BDF system retrieves product images by polling a separate camera service over HTTP.
- Requires: ``REMOTE_CAMERA_BASE_URL``, ``REMOTE_CAMERA_TIMEOUT_S``, ``REMOTE_CAMERA_POLL_INTERVAL_S``.
- **``emulated``**: The main BDF system generates synthetic images internally, without needing physical cameras.
- Can use generated frames based on ``EMULATED_CAMERA_WIDTH``, ``EMULATED_CAMERA_HEIGHT``, ``EMULATED_CAMERA_FPS``.
- Can use an optional source image from ``EMULATED_CAMERA_IMAGE_PATH``.

Recommended Practices for Configuration

To ensure stable and secure operation of your BDF system:

- **Secure Sensitive Data**: Keep all sensitive information (like API keys) exclusively in ``.env`` files.
- **Organize Settings**: Use ``settings.json`` for environment-specific but non-secret settings.
- **Use Profiles**: Utilize ``APP_ENV=debug`` for debug mode instead of directly modifying production values.
- **Docker-Specific Settings**: Apply the ``docker`` profile in ``settings.json`` only for settings that differ when running in a container.

API Reference: Connecting to Your BDF System

This section provides technical details for integrating with your BDF automated inspection system. It outlines the available endpoints and how to interact with them for production monitoring and data access.

Main System Base URL

By default, the main BDF system listens on:

```
```text
```

http://localhost:8000

...

The dedicated host-side camera service (if used in `remote` mode) listens on:

```text

http://localhost:8001

...

GET /

Accessing this endpoint returns the HTML frontend, which is your operator interface from `src/front/index.html`.

Response:

- `200 text/html` (Successful delivery of the web interface)

GET /status

This endpoint provides structured information about the BDF system's current runtime state, including AI initialization, camera readiness, and live feed status. This is crucial for production monitoring.

Example Response Shape:

```json

{

"ai": {

"enabled": true,

"model\_name": "rfdetr",

"phase": "ready",

"message": "RFDETR ready",

"error": null,

"checkpoints": []

},

"camera": {

"ready": true,

```
"message": "Camera ready",

"error": null,

"backend": "local"

},

"stream": {

"has_frame": true,

"mode": "tracked"

}

}

...
```

**Response:**

- `200 application/json` (Successful retrieval of system status)

**GET /capture**

Retrieves the latest JPEG image of a product from the shared frame cache. This is the processed and stitched image ready for inspection or display.

**Success:**

- `200 image/jpeg` (Returns the latest product image)

**Failure Cases:**

- `503` if the camera system is not ready (e.g., cameras initializing).
- `503` if no product image has been produced yet (e.g., during warm-up).

**POST /analyze**

Forwards the latest product image to an external inference service, as defined by `INFERENCE\_URL` in your configuration. This allows for specialized analysis beyond the BDF system's built-in capabilities.

**Behavior:**

- Sends the product image as multipart form data.
- Includes a configured API header (e.g., for authentication) if present.
- Automatically retries on `429` (Too Many Requests) responses from the external service.

**Success:**

- `200 application/json` (Returns the response from the external service)

#### **Failure Cases:**

- `503` if the BDF camera system is unavailable.
- `503` if no product image is available.
- `429` if the external inference service remains at capacity after retries.
- `400` or `503` if the external inference service returns an error.

### **GET /camera/alignment**

Returns the current settings for image stitching alignment (horizontal and vertical offsets, and number of captured strips) for the cameras. This is useful for monitoring and verifying alignment.

#### **Response Shape:**

```

{
 "h_offset": 10,
 "v_offset": 190,
 "num_images": 5,
 "backend": "local"
}

```

#### **Response:**

- `200 application/json` (Returns current alignment settings)
- `503 application/json` if the camera runtime is not initialized or alignment data cannot be read.

### **POST /camera/alignment**

Updates the image stitching alignment settings and resets the internal buffer of stacked frames. This allows operators or integrated systems to fine-tune image composition.

#### **Request Body:**

```

{
 "h_offset": 10,

```

```
"v_offset": 190,
"num_images": 5
}
"
```

`num\_images` is optional and must be at least 1 if provided.

#### **Response Shape:**

```
```json  
{  
  "h_offset": 10,  
  "v_offset": 190,  
  "num_images": 5,  
  "backend": "local",  
  "buffer_reset": true  
}  
"
```

Response:

- 200 application/json (Confirms updated alignment settings)
- 503 application/json if the camera runtime is not initialized or alignment cannot be updated.

GET /color-data

Returns the current buffer of color analysis data for tracked products and then clears this buffer. This provides a snapshot of recent product color information for production monitoring.

Response Shape:

```
```json  
{
 "objects": [
 {
 "id": 1,

```

```
"latest": {},
"history": []
}
]
}
...
```

If detection/tracking is disabled or the AI model is not ready, `objects` will be empty.

## POST /inference

Runs local AI inspection on a specific product image file located on disk. This is useful for analyzing archived images or performing on-demand checks.

### Request Body:

```
```json  
{  
  "feature_name": "inspection",  
  "ImageName": "sample.jpg",  
  "SrcDir": "D:/data/job_001",  
  "Timestamp": "2026-05-12T13:00:00"  
}  
```
```

### Behavior:

- Reads the image from the specified `SrcDir/ImageName`.
- Creates `SrcDir/Inference\_Images/` if it doesn't exist.
- Performs local AI inspection using the configured pipeline.
- Saves an annotated version of the image named `inferenced\_<ImageName>` to `SrcDir/Inference\_Images/`.
- Returns product-shaped color analysis output.

### Success Response Shape:

```
```json  
{
```

```

    "Data": {
    "Products": []
    },
    "Detected": false,
    "Inference_image_path": "D:/data/job_001/Inference_Images/inferenced_sample.jpg",
    "image_name": "sample.jpg"
  }
  ...

```

Failure Cases:

- `404` when the source image file does not exist on disk.
- `503` when the AI model for inspection is still loading or unavailable.

POST /color-analysis

Returns the current contents of the color analysis buffer, formatted for downstream consumption, and then clears the buffer. This provides consolidated color data for integration into BDF's production systems.

Response Shape:

```

`json
{
  "Products": [
    {
      "ID": 1,
      "color_sample": {
        "opening": "square",
        "size": "10x10",
        "positions": [
          {
            "xc": 567,
            "yc": 412

```

```

    }
  ]
},
"confidence": 0.818303,
"color_analysis": {
  "RGB": [130, 225, 79],
  "LChannel": 78.9,
  "CIELAB": [78.9, 42.5, 6.2]
}
}
]
}
'''

```

Response:

- `200 application/json` (Returns the color analysis output)

GET /export_color_csv

Exports the current color analysis buffer as a CSV file. This is useful for long-term data logging and analysis of product color consistency.

Response:

- `200 text/csv`
- `Content-Disposition: attachment; filename=color\_data.csv` (Triggers file download)

GET /delete_all_color_buffer

Clears all buffered color analysis samples from memory. Use this to reset the color history.

Response:

- `200 text/plain`

GET /delete_color_buffer/{obj_id}

Clears buffered color analysis samples for a specific tracked product object ID. This allows targeted management of color data.

Response:

- ``200 text/plain``

WebSocket /ws

Provides a live stream of product images to your browser or other clients.

Stream Content:

- **Raw JPEG Frames:** When the camera is active but AI inspection is not yet ready or enabled.
- **AI-Annotated JPEG Frames:** Once AI inspection is active, frames will include detection and tracking annotations.

Text Messages:

The server sends text messages in cases where frames cannot be provided, such as:

- ``warming_up``: System is starting up.
- Camera error text: Indicates an issue with camera operation.

AI Checkpoints

The ``/status`` endpoint exposes the progress of AI initialization through several checkpoints. These help you monitor when automated inspection will be fully operational.

Current AI checkpoint keys:

- ``pipeline``: Overall AI processing flow.
- ``providers``: AI model dependencies.
- ``weights``: AI model data loading.
- ``tracker``: Object tracking component.
- ``ready``: Indicates all AI components are operational.

Each checkpoint includes:

- ``key``: Unique identifier for the checkpoint.
- ``label``: Human-readable description.
- ``state``: Current status (e.g., ``pending``, ``active``, ``done``, ``error``).
- ``message``: Detailed status message.

Dedicated Camera Unit API

The service started by ``camera_main.py`` (the dedicated camera unit) provides a camera-only API on port ``8001``. This is used by the main BDF system when in ``remote`` camera mode.

GET /status

Returns the camera unit's operational status:

```
```json
```

```
{

 "camera": {

 "ready": true,

 "message": "Camera ready",

 "error": null,

 "backend": "local"

 },

 "stream": {

 "has_frame": true,

 "mode": "raw"

 }

}

"
```

#### GET /camera/alignment

Returns the image stitching alignment state from the camera unit.

#### POST /camera/alignment

Updates the camera-side stitching offsets and optional `num_images``.

#### GET /capture

Returns the latest raw JPEG product image from the camera unit.

#### WebSocket /ws

Streams raw JPEG product images directly from the camera unit.

## Deployment Options: Setting Up Your BDF Automated Inspection System

This guide outlines different ways to deploy and run your BDF automated inspection system. Choose the option that best fits your operational environment and hardware setup.

### Baseline Setup

Before deploying your BDF system in any configuration, ensure these steps are completed:

- **Virtual Environment:** Create a Python virtual environment to manage dependencies.
- **Install Dependencies:** Install all required libraries using `pip install -r requirements.txt``.

- **Environment File:** Create an `.env` file based on `.env.example` and populate it with your specific API keys and settings.
- **Review Settings:** Check `settings.json` to confirm the desired operational profile: `production`, `debug`, or `docker`.

### Option 1: Integrated System Setup (Single-Process Local Deployment)

Use this option when the same Windows machine hosting the main BDF system has direct access to the Basler cameras. This is the simplest configuration for a self-contained system.

#### How to Run:

```

powershell

python .\main.py

```

#### Recommended Settings (in `settings.json` or `.env`):

- `CAMERA_MODE=local`: The main BDF system will directly control the cameras.
- `DETECTION_AND_TRACK=true` or `false`: Enable or disable AI inspection based on your production needs.

#### This is the preferred setup for BDF when:

- Pylon software and drivers are installed locally on the host machine.
- The host machine needs to manage both camera capture and AI processing.
- Network separation between camera and processing is not required.

### Option 2: Testing and Training Mode (Debug or Hardware-Free Deployment)

Choose this option for testing the user interface, API integrations, or AI workflows without requiring physical cameras.

#### How to Run:

```

powershell

$env:APP_ENV="debug"

python .\main.py

```

#### Recommended Settings (in `settings.json` or `.env`):

- `CAMERA_MODE=emulated`: The BDF system will generate synthetic images.
- Optional: `EMULATED_CAMERA_IMAGE_PATH`: Point this to a readable image file if you want deterministic frames for testing (e.g., specific product defects).

### Option 3: Distributed System Setup (Split Camera and App Deployment)

This option is ideal when your camera hardware needs to remain physically close to the production line for optimal image acquisition, while the main BDF system (with its web interface and AI) runs on a separate machine.

#### Camera Host (On the Production Line)

Run the dedicated camera service on the machine that has direct access to the Basler cameras.

##### How to Run:

```
""powershell
```

```
python .\camera_main.py
```

```
""
```

This starts the camera-only service on port `8001`.

#### Main App Host (Separated Processing Unit)

Run the main BDF application on a separate machine (or a different process on the same machine) that will consume images from the camera host.

##### Recommended Settings (in `settings.json` or `.env`):

- `CAMERA\_MODE=remote`: The main BDF system will poll the separate camera service.
- `REMOTE\_CAMERA\_BASE\_URL=http://<camera-host>:8001`: Replace `` with the network address (IP address or hostname) of the camera host machine.

##### Then start:

```
""powershell
```

```
python .\main.py
```

```
""
```

### Option 4: Containerized Deployment for Main System (Docker)

The provided `docker-compose.yml` is intended for deploying the main BDF application in a container, not for direct Basler camera control (which typically requires direct hardware access on the host).

##### Recommended Pattern for BDF:

- Your Windows host machine runs the `camera\_main.py` (as described in Option 3, Camera Host).
- A Docker container runs the main BDF application.
- The main BDF application in the container uses `CAMERA\_MODE=remote`.
- `REMOTE\_CAMERA\_BASE\_URL=http://host.docker.internal:8001`: `host.docker.internal` allows the Docker container to reach services running on the host machine.

## How to Start (from the directory containing `docker-compose.yml`):

```
``powershell
```

```
docker compose up --build
```

```
``
```

## Environment Expectations

### Local Camera Mode

- **Pylon Installation:** Basler Pylon software and drivers must be correctly installed on the host machine.
- **Camera Access:** The BDF system process must have the necessary permissions to open and control both Basler cameras.
- **Configuration Match:** Camera trigger settings and pixel formats in your configuration must match the actual camera hardware setup.

### Remote Camera Mode

- **Camera Service Availability:** The dedicated camera host must expose its API endpoints (`/status`, `/capture`, and `/camera/alignment`) over the network.
- **Network Connectivity:** Stable network connectivity between the main BDF app host and the `REMOTE\_CAMERA\_BASE\_URL` is critical.
- **Frame Freshness:** The timeliness of product images depends on the configured remote polling interval.

### Emulated Mode

- **No Hardware Required:** This mode requires no camera hardware, making it the safest option for frontend development and integration testing.

## Verification Checklist

After deploying your BDF system, follow these steps to verify its operation:

1. **Check System Status:** Call `GET /status` from your web browser or an API tool.
2. **Confirm Camera Readiness:** Verify that `camera.ready` in the `/status` response becomes `true` for your chosen camera mode.
3. **Verify Image Capture:** Confirm that `GET /capture` successfully returns a JPEG product image.
4. **Browser Interface:** Ensure the browser page on `/` loads correctly and displays refreshing product images.
5. **Live Stream:** Check that WebSocket streaming on `/ws` is functioning as expected.
6. **AI Readiness (If Enabled):** If AI inspection is enabled, wait for `ai.phase` in the `/status` response to become `ready`.

7. **Test AI/Color Endpoints:** Once AI is ready, test `POST /analyze`, `POST /inference`, and the color analysis endpoints.

8. **Remote Mode Specific Check:** If using `remote` camera mode, verify that the dedicated camera service on port `8001` also returns a healthy `/status` response.

## Operational Notes

- **System Resilience:** The BDF system is designed to be tolerant of camera startup failures; it will remain online for diagnostics and other functions even if cameras are temporarily unavailable.
- **AI Startup Time:** Be aware that AI model initialization may take noticeably longer than the server's initial startup.
- **Local Inference Files:** The `POST /inference` endpoint requires access to specific filesystem paths on the machine where the main BDF application is running.

## Core Technologies: Building Blocks of Your BDF System

Your BDF automated inspection system is built upon three internal software libraries. These are embedded directly within the system and provide the core functionalities for camera imaging, AI inspection, and color analysis.

### Overview

- **Camera Library:** Handles image acquisition, stitching, and preparation.
- **AI Library:** Manages automated inspection, tracking, and annotation.
- **Color Library:** Provides color analysis and data buffering.

### Camera Library

**Location:**

- `src/libs/racer2\_camera`

**Primary Purpose for BDF:**

- **Basler Device Discovery:** Identifying and connecting to your Basler cameras.
- **Camera Configuration:** Setting up camera parameters for optimal image capture.
- **Line-Scan Acquisition:** Capturing high-speed line-scan images of your products.
- **Image Processing:** Vertically stacking image strips and horizontally stitching images from multiple cameras.
- **JPEG Production:** Preparing processed product images in JPEG format for efficient consumption.

**Key Files:**

- `src/libs/racer2\_camera/src/config.py`: Camera configuration settings.

- ``src/libs/racer2_camera/src/line_camera.py``: The main logic for camera operation and image pipeline.

**Used By:**

- ``src/server/local_camera_runtime.py``: For direct camera control by the main BDF system.
- ``src/server/camera_service.py``: For the dedicated camera unit.

## AI Library

**Location:**

- ``src/libs/ai``

**Primary Purpose for BDF:**

- **AI Model Selection:** Choosing between different AI models for inspection.
- **Asynchronous Model Initialization:** Loading AI models in the background to ensure faster system startup.
- **Live-Frame Inspection:** Performing AI analysis on streaming product images.
- **Single-Image Inspection:** Analyzing individual product image files on demand.
- **Annotation Overlay:** Adding visual markers (e.g., bounding boxes, labels) to product images based on AI detections.
- **Optional Tracking:** Integrating object tracking to monitor detected anomalies over time.

**Key Files:**

- ``src/libs/ai/config.py``: AI-specific configuration settings.
- ``src/libs/ai/pipeline.py``: The main AI processing pipeline logic.
- ``src/libs/ai/src/models/rfdetr.py``: RF-DETR model implementation.
- ``src/libs/ai/src/models/yolo.py``: YOLO model implementation.
- ``src/libs/ai/src/trackers/sort.py``: SORT tracking algorithm implementation.

**Runtime Selection:**

- If ``AI_MODEL_NAME=rfdetr``, the RF-DETR inspection path is used.
- For any other ``AI_MODEL_NAME``, the YOLO detection plus SORT tracking path is used.

**Used By:**

- ``src/server/app.py``: The main BDF application for AI inspection.

## Color Library

**Location:**

- ``src/libs/color``

**Primary Purpose for BDF:**

- **Per-Object Color Extraction:** Analyzing colors around detected and tracked product anomalies.
- **Rolling Object History:** Maintaining a short history of color data for each tracked object.
- **Color Conversion:** Providing helper functions for converting colors between different formats (RGB, XYZ, Lab, hex).

#### Key Exports:

- `ColorAnalyze``: Main class for color analysis.
- `ColorBuffer``: Manages the buffer of color data.
- `rgb_to_xyz`, xyz_to_lab`, rgb_to_hex`, hex_to_rgb`, hex_to_lab``: Utility functions for color space conversion.

#### Used By:

- `src/libs/ai/pipeline.py``: Integrated within the AI inspection process to provide color analysis.

### Reuse Notes

These libraries are designed as internal modules for the BDF system. If you plan to reuse them in other projects:

- **Preserve Structure:** Maintain the current folder and import structure.
- **Dependencies:** Ensure all required third-party Python libraries are carried over.
- **Runtime Assumptions:** Validate any camera or AI runtime assumptions within your new host environment.

## Troubleshooting Guide: Resolving Common Issues for BDF

This section provides solutions to common operational issues you might encounter with your BDF automated inspection system.

### No Product Images Appearing (Server Starts But No Frames Arrive)

#### Check:

- **System Status:** Call `GET /status``.
- **Camera Readiness:** See if `camera.ready`` is `false``.
- **Live Feed Messages:** Check if the WebSocket (`/ws``) is receiving text messages like `warming_up`` instead of images.

#### If using `local` camera mode:

- **Pylon Installation:** Verify that Basler Pylon software is correctly installed.
- **Camera Connection:** Confirm both Basler cameras are physically connected and powered on.

- **Camera Access:** Ensure the BDF system process has the necessary permissions to access the cameras.

**If using `remote` camera mode:**

- **Base URL:** Verify the `REMOTE\_CAMERA\_BASE\_URL` setting for typos.
- **Camera Service Status:** Confirm the dedicated camera service (on the camera host) is running.
- **Remote Capture:** Check if `GET <REMOTE\_CAMERA\_BASE\_URL>/capture` returns a JPEG image from the camera host.

**If using `emulated` camera mode:**

- **Mode Setting:** Verify `CAMERA\_MODE=emulated` in your configuration.
- **Source Image:** If `EMULATED\_CAMERA\_IMAGE\_PATH` is set, ensure it points to a readable image file.

## Camera Not Operating (Camera Busy Or Unavailable)

**Symptoms:**

- `/status` reports `camera not ready`.
- Logs mention `camera initialization failure` or `busy hardware`.

**What to Check:**

- **Close Other Applications:** Ensure Pylon Viewer or any other software using the cameras is closed.
- **Previous Instances:** Confirm no older copies of the BDF service are still running in the background.
- **Restart Host:** If the camera driver appears stuck, a host machine restart might resolve the issue.

## Remote Camera Feed Not Updating (Remote Camera Mode Does Not Refresh)

**Check:**

- **Base URL Accuracy:** Ensure `REMOTE\_CAMERA\_BASE\_URL` has no typos.
- **Port Exposure:** Verify the camera host exposes port `8001`.
- **Remote Status:** Confirm `GET <REMOTE\_CAMERA\_BASE\_URL>/status` returns a `200` response.
- **Network Reachability:** Ensure the main BDF application can reach the camera host from its environment (e.g., check firewall rules).

**Common Causes:**

- **Docker Network:** If using Docker, the container might not be able to reach the host URL (e.g., using an incorrect IP or hostname).
- **Firewall:** Windows firewall on the camera host might be blocking port `8001`.

- **Remote Capture Failure:** The camera host might be up, but its own `/capture` endpoint could be failing.

## Live Feed Shows Messages Instead of Images (WebSocket Stream Shows Text Instead Of Images)

This behavior is expected in certain conditions:

### This is normal when:

- The camera system is warming up.
- Camera initialization has failed.
- In `remote` mode, the system has not yet received a valid image from the camera service.

### The text messages usually indicate:

- Camera status updates (e.g., `camera error`).
- `warming\_up` messages.

## Automated Inspection Not Starting (AI Never Becomes Ready)

### Possible Causes:

- Invalid Roboflow credentials.
- Long initialization time for large AI models.
- Unsupported local runtime environment (e.g., missing GPU drivers).

### What to Verify:

- **System Status:** Call `GET /status` and check the `ai` section.
- **Credentials:** Confirm your `.env` file contains valid AI credentials (e.g., `ROBOFLOW\_API\_KEY`).
- **Model Name:** Verify `AI\_MODEL\_NAME` in your configuration is set to the intended model.

## On-Demand Image Analysis Fails (POST /inference Returns 404)

### Check:

- **Source Directory:** Confirm `SrcDir` points to an existing directory on the machine running the main BDF app.
- **Image Name:** Verify `ImageName` is correct.
- **Full Path:** Ensure the combined path `SrcDir/ImageName` points to an existing and readable image file.

### Remember:

- This endpoint reads product images directly from the local disk of the machine running the main BDF application.
- It does not fetch images from remote network locations.

## Color Analysis Data Is Missing (Color Endpoints Return Empty Data)

### Possible Reasons:

- **Detection/Tracking Disabled:** `DETECTION\_AND\_TRACK=false` in your configuration.
- **AI Model Not Ready:** The AI model is still loading or has failed.
- **No Tracked Objects:** No products have been detected and tracked since the last buffer clear.
- **Buffer Already Cleared:** A previous `GET /color-data` or `POST /color-analysis` API call already emptied the buffer.

## External Analysis Connection Fails (External Inference Endpoint Fails)

### Possible Causes:

- `INFERENCE\_URL` is incorrect.
- `INFERENCE\_API\_HEADER` is incorrect.
- `INFERENCE\_API\_KEY` is incorrect or missing.
- The external inference service is unavailable or unreachable.

### What to Verify:

- **Configuration Files:** Check your `.env` and `settings.json` files for correct settings.
- **Network Reachability:** Confirm that the machine running the main BDF app can reach the external inference service over the network.

## Incorrect Colors or Image Errors (Pixel Format Problems)

### Symptoms:

- Wrong colors in the output product images.
- Conversion errors during image capture.
- OpenCV errors related to YUV conversion.

### What to Verify:

- **Pixel Format Setting:** Ensure `CAMERA\_PIXEL\_FORMAT` in your configuration accurately matches the actual pixel output format of your Basler cameras.
- **Pylon Configuration:** If camera configuration is managed externally, set `CAMERA\_CONFIG\_BY\_PYLON=true`.
- **BGR8 Usage:** Use `BGR8` as the pixel format when your camera is already configured to emit OpenCV-ready BGR images.

## Step-by-Step Diagnostic Check (General Diagnostic Sequence)

Follow these steps for a general diagnosis of your BDF system's operation:

1. **Check Overall Status:** Call `GET /status`.
2. **Confirm Camera Ready:** Verify that `camera.ready` in the status response is `true`.

3. **Verify Image Capture:** Confirm that `GET /capture` successfully returns a JPEG image.
4. **Check Browser Interface:** Ensure the browser page on `/` loads and displays live product images.
5. **Wait for AI (If Enabled):** If AI inspection is enabled, wait for `ai.phase` in the status response to become `ready`.
6. **Test AI/Color Functions:** Once AI is ready, test `POST /analyze`, `POST /inference`, or WebSocket streaming (`/ws`) for annotated frames and color data.